Activity 4: Wallet and Digital Signature Lab

Maphoka Mahlomola | 224028650

Introduction

Blockchain technology uses public key cryptography to prove ownership of digital property and to ensure that transactions are not tampered with. An essential characteristic of this technology is the use of asymmetric cryptography, where each participant has a private key and a public key. The private key is used to authorise transactions using digital signatures, while the public key can be used by third parties to validate those signatures.

In this project, I implemented a simplified version of a blockchain wallet and transaction signing. I created two participants, Alice and Bob, each with their own Elliptic Curve Digital Signature Algorithm (ECDSA) key pairs generated using the secp256k1 elliptic curve. Their public keys were serialized and converted into simplified wallets. Then Alice created a transaction transferring 10 DemoCoin to Bob, signed it with her private key, and the signature was verified using Alice's public key. Finally, the transaction was altered by changing its amount from 10 to 1000 DemoCoin, and it showed that the original digital signature could no longer be validated.

The objective of this implementation is therefore to demonstrate the relationship between private keys, public keys, wallet addresses, digital signatures, and transaction integrity.

Key Generation and Public Key Cryptography

The first step in the implementation involves creating separate ECDSA key pairs for Alice and Bob. This is done using the Python cryptography library in conjunction with the secp256k1 elliptic curve. For each of the participants, a private key is created, and a public key is then derived from the private key.

Theoretically speaking, given the private key is denoted as (d), the public key can be expressed in terms of:

$$[\, Q = dG \,]$$

Here, (G) refers to the generator point on the selected elliptic curve, while (Q) is the resultant public key.

What makes this particular system a secure one is the fact that although it is practically possible to derive the public key from the private key, the inverse cannot be done under any circumstances. Hence, both Alice and Bob can publicly reveal their public keys without compromising their private keys.

In the case of the Python implementation, Alice's private key is first created using the secp256k1 elliptic curve. Subsequently, the public key is derived from the private key. The same is true for Bob. The private keys are not printed out by the program. This follows the very basic premise of blockchain wallet security, that of the private key ownership implying transaction authorisation.

Public-key Serialization and Wallet Address Derivation

The generated public keys start off as cryptographic objects. To obtain wallet addresses, they need to first be serialized in the form of a byte array according to the X9.62 uncompressed point format. Once the public keys are serialized, they can be subjected to cryptographic hash functions.

A simplified wallet address can be obtained according to the following procedure:

Firstly, the serialized public key is hashed with SHA-256:

[$H_1 = \text{SHA256}(\text{PublicKey})$]

The result is then used in RIPEMD-160:

[$H_2 = \text{RIPEMD160}(H_1)$]

After which the result from RIPEMD-160 is then hex encoded and prefaced with WALLET_ to generate a simple example of an address.

The above addresses are simple representations and are not production Bitcoin addresses. In a real-world case, the Bitcoin address will involve other elements like versioning, checksums and an encoding algorithm. Nonetheless, the above simple method is enough to show how it is possible to generate a wallet address from a public key without exposing the corresponding private key.

This is possible since Alice and Bob have generated different private keys and thus different public keys and wallets addresses.

Transaction Creation and Digital Signing

After creation of the wallet addresses, a transaction is made where Alice sends 10 DemoCoin to Bob. The transaction comprises the address of the sender, the address of the recipient, the amount of the transaction and the currency of the transaction.

However, before signing of the transaction, it should be serialized, meaning converted into a sequence of bytes. This is achieved by JSON serialization with sort_keys set to true and using compact separators. Serialization is required because of the deterministic nature of the transaction. It is important that the transaction is deterministic since a digital signature is tied to the exact byte representation of the message being signed.

Then, the transaction is digitally signed using Alice's private key by implementing ECDSA and SHA-256. The conceptual signing process may be represented as follows:

[S = Sign (SK_A, H(T))]

where: S - Digital Signature

SK_A – Alice's Private Key

T – The transaction

H(T) – The SHA-256 hash of the transaction

The private key is thus used to create proof of authorship of the transaction. Importantly, the private key is not transmitted through the process of signing and making the transaction.

Signature Verification and Transaction Integrity

Following the signature of the transaction by Alice, the public key of Alice is used to validate the signature. Validation can be expressed logically as:

[Verify(PK_A, S, T) = True]

where (PK_A) is Alice's public key, (S) is the signature and (T) is the original transaction.

Successful validation implies that the signature is validated against the original transaction and the related public key. In terms of the code implementation, the program will give a successful validation message for the original transaction.

The last step demonstrates the validity of the transaction through alteration of the original transaction. The amount will be altered from 10 to 1000, this altered transaction will be serialized in the same manner as the original transaction using the deterministic JSON method. But the program attempts to validate this altered transaction using the signature for the original transaction.

Since the transaction has been altered, the signature verification on the original signature will fail:

[Verify (PK_A, S, T') = False]

Such a situation shows one of the key characteristics of digital signatures' integrity. Modification of any part of the transaction leads to an invalid signature. It is impossible for the attacker to change the transaction amount from 10 to 1000 without making modifications to Alice's original valid signature.

Authentication is another feature of digital signatures. The successful verification of a signature proves that the signature was made with a private key that corresponds to the public key in the verification process.

Conclusion

This project implemented a simple blockchain-inspired wallet and transaction authorisation system in Python and ECDSA. Individual secp256k1 key pairs were generated for both Alice and Bob, and then their respective public keys were serialized and hashed to generate simplified wallet addresses.

The transaction from Alice to Bob was made using deterministic JSON and signed with Alice's private key. This signature was successfully verified by Alice's public key, illustrating the link between asymmetric keys and digital signatures. Once the transaction was altered by changing the amount from 10 to 1000 DemoCoin, the signature could not be verified since it matched only the initial transaction.

All in all, this project has shown how public-key cryptography helps authenticate and validate transactions within blockchain-based networks. The private key is used to authorise the transaction through digital signing, and then the public key is used to verify the signature for other network participants. Most importantly, the mutation test illustrates how a valid digital signature cannot be used for verifying an altered transaction. This project implemented a simple blockchain-inspired wallet and transaction authorisation system in Python and ECDSA. Individual secp256k1 key pairs were generated for both Alice and Bob, and then their respective public keys were serialized and hashed to generate simplified wallet addresses.

Github link: https://github.com/Maphoka-M/Assignment-4